Mentimeter

Laborator 11

Legare și execuție

Mentimeter

Puterea generativa a limbajului

- putem folosi scopuri pentru a obtine rezulttate de orice forma
- ex: myConcat([1], L, [1, 5, 7]) - care sunt valorile lui L pentru care scopul este adevarat?
- se cauta instantierile variabilelor incluse

Mentimeter

```
1 ?- length(L, 3), member(1, L), member(2, L).
L = [1, 2, _] ;
L = [1, _, 2] ;
L = [2, 1, _] ;
L = [_, 1, 2] ;
L = [2, _, 1] ;
L = [_, 2, 1] ;
false.
```

Exemplu

Mentimeter

Explicatii

- member(E, L) este true daca elementul E apartine listei L
- length(L, N) este true daca L are N elemente
- length(L, 3), member(1, L), member(2, L) => toate listele cu 3 elemente care contin 1 si 2 (pe orice pozitie)

Mentimeter

Problema colorarii

- N noduri si M culori
- trebuie sa coloram fiecare nod a.i. niciun nod sa nu aiba aceeasi culoare cu vecinul sau

Mentimeter

```
%% includes(?L1, ?L2) - toate elementele din L1 sunt incluse in L2
includes([], _).
includes([X | Y], L2) :- member(X, L2), includes(Y, L2).

%% all_numbers(?N, ?L) - L o sa contina toate valorile de la 1 la N in ordine descrescatoare
all_numbers(0, []).
all_numbers(N, [N | L]) :- N > 0, N1 is N - 1, all_numbers(N1, L).

%% safe verifica ca nu exista un nod care sa aiba aceeasi culoare cu vecinul sau,
%% dar aici nu ne complicam si cu asta, importanta e generarea
safe(_).
solve(N, M, Sol) :- all_numbers(M, Colours), length(Sol, N), includes(Sol, Colours), safe(Sol).
```

Backtracking 1



Mentimeter

Explicatii 1

- (nu ne pasa exact cum verificam ca e corecta solutia, important aici e cum se genereaza)
- pentru fiecare nod ne trebuie o culoare => Sol o sa fie o lista cu N valori
- cum stim ce culoare? numerotam culorile de la 1 la N

Mentimeter

Explicatii 2

- trebuie sa coloram nodurile numai cu acele culori => culorile folosite trebuie sa fie incluse in cele disponibile
- trebuie sa verificam ca solutia este corecta => safe(Sol)

Mentimeter

Ce e prost?

Generam toate variantele posibile fara sa le dam fail celor care sunt deja proaste (daca am colorat 2 noduri vecine cu aceeasi culoare atunci solutia e deja gresita si nu mai trebuie sa le coloram si pe restul)

Mentimeter

Solutie + templates

- luam cazul particular N = 7 (7 tari), M = 3 (3 culori: r, g, b)
- ca sa stim exact ce culoare este pentru ce nod putem folosi un template
- template([1/, *2/*, 3/, *4/*, 5/, *6/*, 7/_]).
- a/b: nodul a are culoarea b (puteam sa le legam si altfel)

Mentimeter

```
template([1/_, 2/_, 3/_, 4/_, 5/_, 6/_, 7/_]).

%% safe primeste acum noua colorare (nod + culoare) si solutia partiala si verifica
%% daca nu e deja o solutie proasta
solve_recursive([]).
solve_recursive([Nod/Col | L]) :- solve_recursive(L), member(Col, ['r', 'g', 'b']), safe(Nod/Col, L).

solve(Sol) :- template(Sol), solve_recursive(Sol).
```

Backtracking 2



Mentimeter

Negatia \+

- avem un predicat p care poate contine sau nu variabile
- \+p(argumente) este adevarat daca predicatul pe acele argumente nu este satisfiabil

Mentimeter

```
student(andrei). student(marius). lazy(marius).
p1(X) :- student(X), \+ lazy(X). %% X = andrei
p2(X) :- \+ lazy(X), student(X). %% false
```

Ordinea conteaza

Mentimeter

p1(X) :- student(X), \+ lazy(X).

- student(X). => X poate fi andrei sau marius
- \+ lazy(X) => care sunt valorile lui X (**andrei/marius**) pentru care lazy(X) este fals?
- numai andrei

Mentimeter

p2(X) :- \+ lazy(X), student(X).

- \+ lazy(X) si inca nu stiu nimic de X => \+ lazy(X) este false pentru ca exista un X = marius a.i. lazy(X) este true
- nu mai ajung sa verific ca student(X) este adevarat pentru ca \+lazy(X) este false si se intoarce false

Mentimeter

Predicatul cut !

- ce e in stanga lui ! **nu mai poate fi resatisfacut**
- ce e in dreapta lui da si se poate aplica backtracking

Mentimeter

```
min1(X, Y, Min) :- X < Y, Min = X.
min1(X, Y, Min) :- X >= Y, Min = Y.

%% daca rulez min(2, 3, Min). din terminal o sa afiseze Min = 2; false
%% stiu sigur ca indiferent ce valori au X si Y voi avea un singur minim si atunci
%% nu are sens sa incerc resatisfacerea scopului
min2(X, Y, Min) :- X < Y, !, Min = X.
min2(X, Y, Min) :- X >= Y, Min = Y.
```

Exemplu min



Mentimeter

Green cut vs red cut

- Green cut: daca scot ! executia este inca corecta
- Red cut: fara ! nu mai obtin solutia corecta

Mentimeter

```
%% daca rulez min(2, 3, Min). din terminal o sa afiseze Min = 2; false
%% stiu sigur ca indiferent ce valori au X si Y voi avea un singur minim si atunci
%% nu are sens sa incerc resatisfacerea scopului
min2(X, Y, Min) :- X < Y, !, Min = X.
min2(X, Y, Min) :- X >= Y, Min = Y.

%% daca X < Y atunci voi incerca sa gasesc Min = X, dar nu voi mai folosi
%% a doua regula de la min pentru a resatisface scopul (de la !)
%% => daca ajung la a doua regula inseamna ca X >= Y obligatoriu
%% => nu mai am nevoie sa verific si pot sa simplific regula 2
min3(X, Y, Min) :- X < Y, !, Min = X.
min3(X, Y, Min) :- Min = Y.
```

min2 - greet cut, min3 - red cut

Mentimeter

```
p(a).
p(b).
p(A/B) :- q(A), !, t(A/B).
p(d).

q(a).
q(b).
q(c).

t(a/a).
t(a/b).
t(b/c).
t(b/d).
```

Exemplu lab

Mentimeter

Ce afiseaza p(X).?

- X = a ; X = b ; X = a/a ; X = a/b.
- primele 2 valori sunt luate din primele 2 reguli de la p(stim ca p(a) si p(b) sunt true)
- mai departe aplicam regula 3 p(A/B) :- q(A), !, t(A/B).

Mentimeter

Explicatii continuare

- trebuie sa satisfacem q(A) si o prima legare ar fi A la a (prima regula de la q)
- dupa avem ! deci nu se vor mai cauta alte solutii pentru ce am legat deja A = a, X = A/B
- dupa ! se incearca satisfacerea lui t(a/B) care admite ca solutii B = a/b
- deci solutiile finale sunt: a, b, a/a, a/b

Mentimeter

15. Câte variante de satisfacere a scopului `a(X, Y, Z)` există pentru programul de mai jos?

```
p(1). p(2). p(3).
q(3). q(4).
r(1). r(2). r(3).
a(X, Y, Z) :- p(X), !, \+ q(X), q(Y), !, r(Z).
```

(a) 3
(b) 0
(c) 6
(d) 12

Exemplu mai dubios din grile

?

Mentimeter

Metapredicate

- predicate care primesc scopuri ca argumente
- findall, bagof, setof, forall

Mentimeter

findall(+Template, +Goal, -Bag)

pentru fiecare satisfacere a Goal-ului cream
o instanta a Template-ului pe care o
adaugam in Bag

Mentimeter

```
even(L, Even) :- findall(X, (member(X, L), X mod 2 =:= 0), Even).
pairs(L, Pairs) :- findall((X, Y), (member(X, L), member(Y, L), X \= Y), Pairs).

%% exemplu mai fancy:
%% - caut toti divizorii de la 2 la sqrt(N) (numlist(A, B, L) - L contine toate
%%   numerele intregi de la A la B, dar si A si B trebuie sa fie numere intregi)
%% - daca nu exista (length(Divisors, 0)) atunci numarul e prim
isPrime(N) :-
    findall(D, (S is round(sqrt(N)), numlist(2, S, All), member(D, All), mod(N, D) =:= 0), Divisors),
    length(Divisors, 0).
```

Exemple



Mentimeter

bagof(+Template, +Goal, -Bag)

asemanator cu findall dar se face cate o instanta a lui Bag pentru fiecare instantiere diferita a variabilelor libere (variabile libere = variabilele care nu apar in template)

Mentimeter

```
%% pentru fiecare valoare X din L se face o lista cu toate valorile Y mai mari decat X
biggerValues(L, Bigger) :- bagof(Y, (member(X, L), member(Y, L), X < Y), Bigger).

%% exemplu: biggerValues([20, 4, 5, 3], L). afiseaza:
%% L = [20, 4, 5] ;
%% L = [20, 5] ;
%% L = [20].
```

Exemplu



Mentimeter

setof(+Template, +Goal, -Bag)

la fel ca bagof, dar aici fiecare instanta a lui
Bag este un set (multime fara duplicate)
sortata

Mentimeter

```
%% pentru fiecare valoare X din L se face o lista cu toate valorile Y mai mari decat X
biggerValuesSets(L, Bigger) :- setof(Y, (member(X, L), member(Y, L), X < Y), Bigger).

%% exemplu: biggerValuesSets([20, 4, 5, 3, 20], L). afiseaza
%% L = [4, 5, 20] ;
%% L = [5, 20] ;
%% L = [20].
```

Exemplu

Mentimeter

forall(+Cond, +Action)

verica daca se pot indeplini conditiile din
Action pentru orice legare din Cond

Mentimeter

```
onlyEvens(L) :- forall(member(X, L), mod(X, 2) =:= 0).

%% ceva putin mai fancy: avem o lista de liste si vrem sa verificam fiecare lista
%% are numai elemente pozitive
onlyPositives(All) :- forall(member(List, All), forall(member(X, List), X > 0)).
```

Exemplu